Product Return and Refund Analytics System for E-Commerce Platforms using MongoDB and HDFS

Duaa Awan(B23F0001DS009)

And

SHAHBAZ ALI KHAN(B23F0001DS004)

➢ Dataset Used

I created my own dataset for this project which is ecommerce returns dataset.

➢ Problem Statement

E-commerce platforms such as Amazon and Daraz face millions of dollars in revenue loss every year due to product returns and refund abuse. The core problem is that current return systems cannot detect suspicious return patterns or fraudulent refund claims at scale. This project analyses large-scale e-commerce transaction data using Hadoop and MongoDB to detect fraudulent return behaviour, identify high-risk customers, and calculate the financial impact of returns on business revenue.

➢ Objectives of the Project

To analyse product return patterns across different categories and time periods

To detect customers with suspicious or fraudulent return behaviour using a scoring system

To calculate total revenue loss caused by refunds using Hadoop MapReduce

To identify which product categories have the highest return rates

To determine whether late delivery is a major cause of product returns

To store and query return records efficiently using MongoDB

To process large-scale transaction logs using Hadoop MapReduce batch jobs

➢ Characteristics of the Data

Volume: 10,000 order records covering 3 years (2022–2024). In real-world e-commerce this scales to millions of daily transactions, which justifies the use of Hadoop for distributed processing.

Velocity: Order and return data is generated continuously every day. New transactions are added regularly, requiring batch processing pipelines to handle incoming data on a scheduled basis.

Variety: The dataset contains structured fields (Order_ID, Amount, Dates), categorical fields (Category, Return_Reason, Payment_Method), and optional fields (Return_Date and Return_Reason are empty for non-returned orders). This variety makes MongoDB a suitable storage choice because it handles missing fields naturally.

Veracity: Real e-commerce data contains noise such as missing return dates, duplicate entries, and inconsistent reason labels. Data cleaning is performed before loading into Hadoop and MongoDB.

Value: The processed data produces fraud risk scores per customer, financial loss reports, and category-level return insights that directly support business decision-making.

➢ Big Data Tools & Technologies

Tool	Purpose
Hadoop HDFS	Distributed file storage of the full CSV dataset
Hadoop MapReduce	Batch processing return rate calculation, refund totals, repeat returner detection
MongoDB	NoSQL document database for storing and querying return records
Python	Writing MapReduce mapper and reducer scripts using Hadoop Streaming
PyMongo	Python library to insert CSV data into MongoDB and run aggregation queries

---

➢ Data Ingestion Method

Method: Batch Ingestion

The dataset is a static CSV file containing historical order and return records from 2022 to 2024. It will be ingested in two ways:

Into Hadoop HDFS: hadoop fs -mkdir -p /user/data/returns hadoop fs -put ecommerce_returns_dataset.csv /user/data/returns/

Into MongoDB using Python and PyMongo — a script reads each row of the CSV and inserts it as a JSON document into a MongoDB collection named ecommerce_returns.

➢ Data Storage Plan

Hadoop HDFS: The full CSV dataset is uploaded to HDFS for distributed storage. Hadoop reads this file in parallel blocks during MapReduce jobs. This allows processing of large files that would be too slow on a single machine.

MongoDB: Each order row is stored as a separate JSON document in the ecommerce_returns collection. MongoDB is chosen because return records have optional fields — non-returned orders have no Return_Date or Return_Reason. MongoDB handles missing fields naturally without requiring NULL placeholders like SQL databases do.

➢ Data Processing Approach Method:

Hadoop MapReduce for batch processing + MongoDB Aggregation Pipeline for fraud detection

Hadoop MapReduce Jobs (Python scripts run via Hadoop Streaming):

➢ Cluster / Environment Setup Plan Environment:

Single-node Hadoop cluster in Pseudo-distributed mode on local machine

Setup steps:

- Install Java JDK 8
- Install Hadoop
- Format the HDFS namenode: hdfs namenode -format
- Start HDFS
- Start YARN
- Install MongoDB Community Edition and start the MongoDB service
- Install Python dependencies
- Upload dataset to HDFS
- Load dataset into MongoDB using Python ingestion script
- Run MapReduce jobs using Hadoop Streaming

➢ Expected Outcomes / Insights

Fashion category will have the highest return rate followed by Electronics

Approximately 28–30% of all orders will be returned

Total refund loss will exceed $1.6 million across the dataset

Around 5 to 10% of customers will be flagged as high-risk based on repeat return patterns

Late delivery will show a positive correlation with return rates

Electronics will show the highest average refund amount due to high product prices

A fraud risk report will be generated listing every customer with their risk level and total refund claimed

➢ Real-World Application

This system directly applies to platforms such as Amazon, Daraz, and Shopify:

Fraud prevention teams can use the customer fraud score to flag accounts before approving refunds

Category managers can reduce return rates by improving product descriptions and quality checks for high-return categories

Finance teams can quantify exact revenue lost to returns and set policy limits accordingly

Logistics teams can use delivery time vs return rate analysis to prioritise faster delivery in high-return regions

➢ Project Timeline

WEEK 1 ,2 ,3 WORKING

WEEK 4 FINAL EVALUATION